

Objects

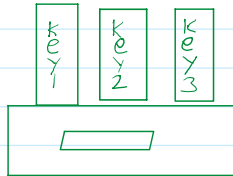
07 September 2026 19:00

As we know from the chapter Data Types, there are eight data types in Javascript. Seven of them are called primitive because their values contain only a single thing (be it a string or a number or whatever).

In contrast, objects are used to store keyed collection of various data and more complex entities. In Javascript, objects penetrate almost every aspect of the language. So we must understand them first before going in depth anywhere else.

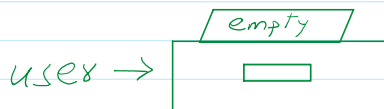
An object can be created with curly braces {...} With an optional list of **properties**. A property is a "key:value" pair where **key** is a string (also called a property name) And a **value** can be anything.

We can imagine an object as a cabinet with signed files. Every piece of data is stored in its file by the key. It's easy to find a file by its name or add or remove a file.



An empty object (empty cabinet) can be created using one of the two syntaxes:

```
let user = new Object(); // "object constructor" syntax
or
let user = {}; // "object literal" syntax
```



Usually, the curly braces {...} are used. That declaration is called an object literal.

Literal and properties

We can immediately put some properties into {... } as "key:value" pairs.

```
let user = {          // an object
  name: "John",       // by key "name" store value "John"
  age: 30              // by key "age" store value 30.
};
```

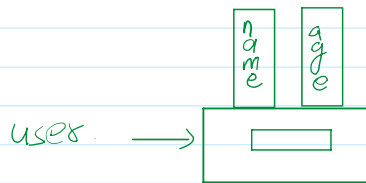
A property has a key (also known as "name" or "identifier") before the colon ":" and the value to the right of it.

In the user object there are two properties:

- The first property has the name "name" and the value "John".
- The second one has the name, "age" and the value 30.

The resulting user object can be imagined as a cabinet with two signed file labeled *name* and *age*.

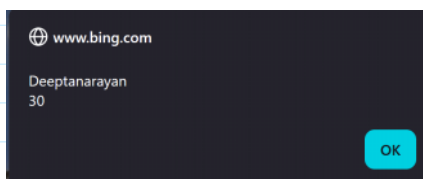
The resulting user object can be imagined as a cabinet with two signed file labeled *name* and *age*.



Property values are accessible using the dot notation.

```
>> let user = {  
  name: "Deeptanarayan",  
  age: 30  
};  
← undefined  
>> user  
← ▶ Object { name: "Deeptanarayan", age: 30 }
```

```
>> alert(user.name)  
← undefined  
>> alert(user.age)  
← undefined  
>> alert(user.name+"\n"+user.age)
```



The value can be of any type. Let's add a boolean one.

```
>> user.isAdmin = true;  
← true  
>> user  
← ▶ Object { name: "Deeptanarayan", age: 30, isAdmin: true }
```

name new property = value
object

```
>> user.isLoggedIn = "Yes";  
← "Yes"  
>> user  
← Object { name: "Deeptanarayan", age: 30, isAdmin: true, isLoggedIn: "Yes" }
```

To remove a property we can use the delete operator:

```
>> delete user.isLoggedIn;  
← true  
>> user  
← Object { name: "Deeptanarayan", age: 30, isAdmin: true }
```

The last property in the list may end with a comma.

```
let user = {  
  name: "John",  
  age: 30,  
};
```

That is called a trailing or hanging comma. Makes it easier to add, remove, move

around properties because all lines become alike.

```
>> let student = {  
  roll_num: 101,  
  name: 'Jones',  
}  
← undefined  
>> student  
← ▶ Object { roll_num: 101, name: "Jones" }  
>> |
```

Square brackets

For multi word properties the dot access doesn't work:

```
//user.likes birds = true;
```

Javascript doesn't understand that it thinks that we address user.likes, and then gives us syntax error when comes across unexpected words.

The dot requires the key to be a valid variable identifier. That implies: contains no space. Doesn't start with a digit and doesn't include special characters (dollar '\$' and '_' are allowed).

There's an alternative "Square bracket notation" that works with any string:

```
let user = {  
  name: "John",  
  age: 30,  
};  
console.log(user);  
user[`likes birds`] = true;  
console.log(user);  
delete user[`likes birds`]  
console.log(user)
```

Output:

```
{ name: 'John', age: 30 }  
{ name: 'John', age: 30, 'likes birds': true }  
{ name: 'John', age: 30 }
```

Now everything is fine. Please note that the string inside the bracket is properly coated. (any type of codes will do).

Square brackets also provide a way to obtain the property name as a result of any expression - as opposed to a literal string - like. From a variable as follows:

```
let user = {  
  name: "John",  
  age: 30,  
};  
console.log(user);  
let key = `likes birds`;  
user[key] = true;  
console.log(user)
```

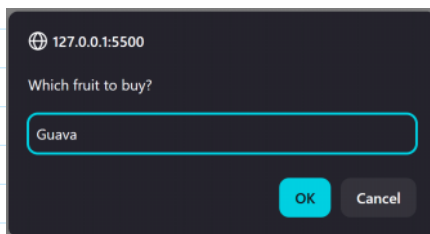
Output:

```
{ name: 'John', age: 30 }  
{ name: 'John', age: 30, 'likes birds': true }
```

Here the variable key may be calculated at run - time or depend on the user input. And then we use it to access the property. That gives us a great deal of flexibility.

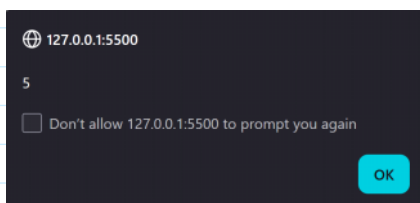
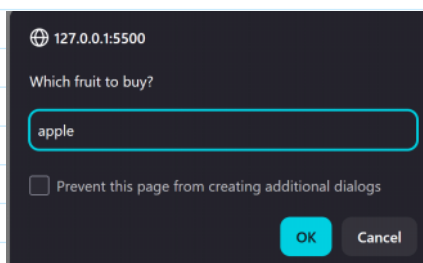
For instance:

```
let fruit = prompt(`Which fruit to buy? `, `apple`);
let bag= {
  // the name of the property is taken from the variable fruit.
  [fruit]: 5,
};
console.log(bag);
```



```
► Object { Guava: 5 }
```

```
let fruit = prompt(`Which fruit to buy? `, `apple`);
let bag= {
  // the name of the property is taken from the variable fruit.
  [fruit]: 5,
};
alert( bag.apple );
```



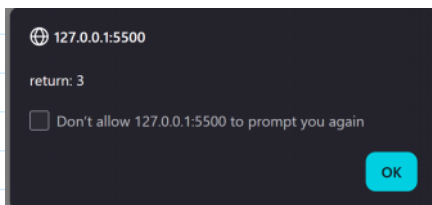
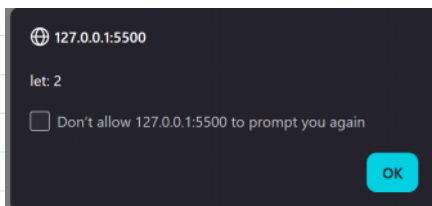
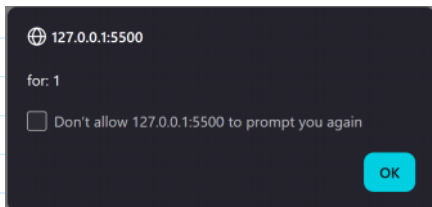
The meaning of a computed property is simple: [fruit] Means that the property name should be taken from fruit.
So if the visitor enters apple bag will become {apple: 5 }.

Property names limitations

As we already know, a variable cannot have a name equal to one of the language - reserved words like `for`, `let`, `return`, etc.

```
let obj = {  
  for: 1,  
  let: 2,  
  return: 3  
};  
alert(obj.for)  
alert(obj.let)  
alert(obj.return)
```

Output:



In short, there are no limitations on property names. They can be any strings or symbols.

Other types are automatically converted to strings.

For instance, a number `0` becomes a string `"0"` when used as a property key.

```
let obj = {  
  for: 1,  
  let: 2,  
  return: 3,  
  0: `test`  
};  
alert(`for: ${obj.for}`);  
alert(`let: ${obj.let}`);  
alert(`return: ${obj.return}`);  
alert(`0: ${obj[0]}`);
```

HTML code:

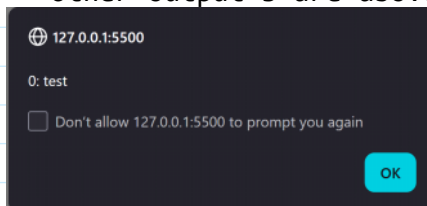
```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Object Two</title>
</head>
<body>
  <script src="../../JS/ObjectsInJS/objectTwo.js"></script>
</body>
</html>

```

Output:

Other output's are above



Property value Shorthand

In real code we often use existing variables as values for property names.
For instance:

```

function makeuser(name, age){
  return {
    name: name,
    age: age,
    // ...other properties
  };
}
let user = makeuser(`Deeptanarayan`, 21);
alert(user.name)

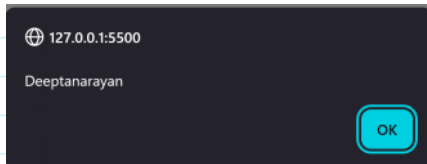
```

HTML code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Object Three</title>
</head>
<body>
  <script src="../../JS/ObjectsInJS/objectThree.js"></script>
</body>
</html>

```



In the example above, properties have the same name as variables. The use case of making a property from a variable is so common that there's a special property value shorthand to make it shorter.

Instead of `name: name` we can just write `name` like this:

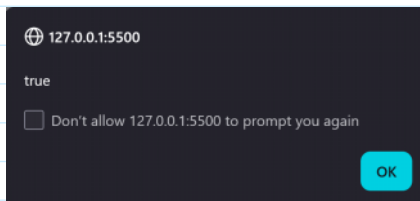
```
function makeuser(name, age){
  return {
    name, //same as name: name
    age, //same as age: age
    // ...other properties
  };
}
```

Property existence test, "in" operator

A notable feature of objects in Javascript compared to many other languages is that it's possible to access any property. There will be no error if the property doesn't exist!

```
let user={};
//true means "no such property"
alert( user.noSuchProperty === undefined);
```

Output:



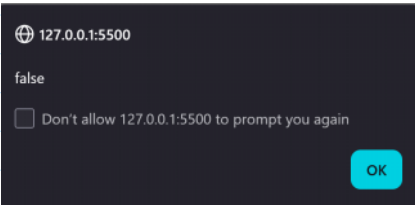
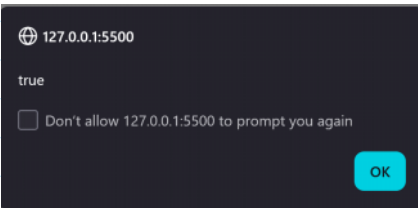
There's also a special operator `"in"` for that.

The syntax is:
"key" in object

For instance:

```
let user={ name:`Deeptanarayan`, age: 20};
alert("age" in user); //true, user.age exists
alert( "blabla" in user); //false, user.blabla doesn't exists
```

Output:

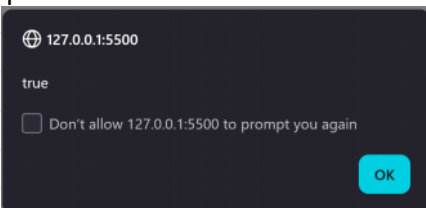


Please note that on the left side of `in` there must be a property name. That's usually a coated string.

If we omit codes, that means the variable should contain the actual name to be tested, for instance:

```
let user = { age: 20 };
let key = "age";
alert(key in user); // true, property exists
```

Output:



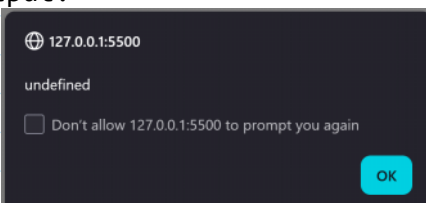
Why does the `IN` operator exist? Isn't it enough to compare against `undefined`?

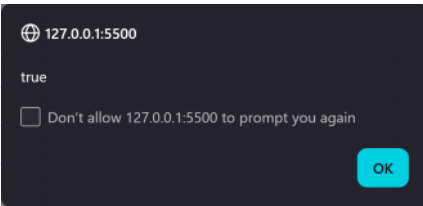
Well, most of the time the comparison with `undefined` works fine. But there's a special case when it fails but `"in"` works correctly.

It's when an object property exist but stores `undefined`:

```
let obj = {
  test: undefined
};
alert(obj.test); // It's undefined, so no such property?
alert("test" in obj); //true, the property does exist!
```

Output:





In the code above the property `obj.test` technically exist. So **the** `in` operator works right.

Situations like this happens very rarely, because undefined should not be explicitly assigned. We mostly use `"null"` for `"unknown"` or `"empty"` values.

The `"for..in"` loop

To walk over all keys of an object, there exist a special form of the loop: `for..in`. This is a completely different thing from the `for (;;)` Construct that we studied before.

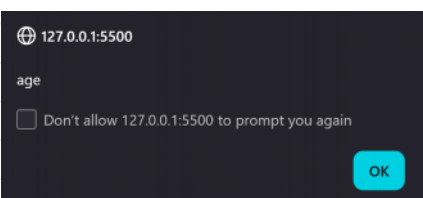
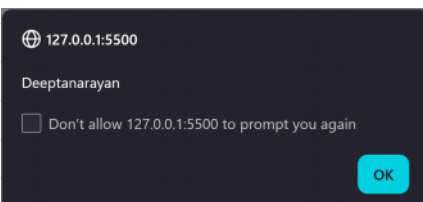
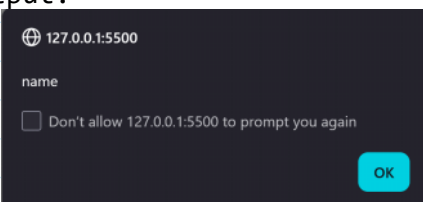
The syntax:

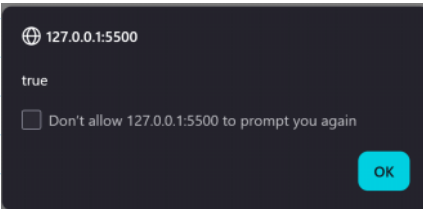
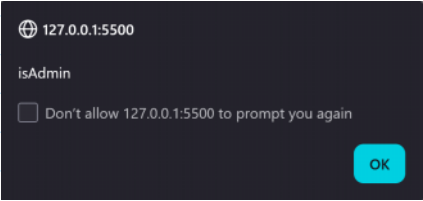
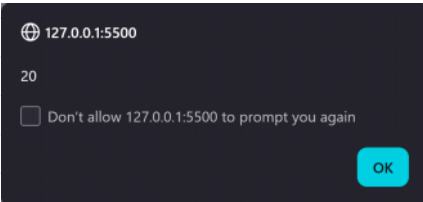
```
for (key in object){  
    // Executes the body for each key among object properties.  
}
```

For instance, let's output all properties of `user`:

```
let user = {  
    name: `Deeptanarayan`,  
    age: 20,  
    isAdmin: true  
};  
for(let key in user){  
    //keys  
    alert(key); //name, age, isAdmin  
    alert(user[key]); //Deeptanarayan, 20, true  
}
```

Output:





Note that all four constructs allow us to declare the looping variable inside the loop, like `let key` here.

Also we can use. Another variable name here instead of `key`. For instance, `"for( let prop in obj) "` is also widely used.

Ordered like an object

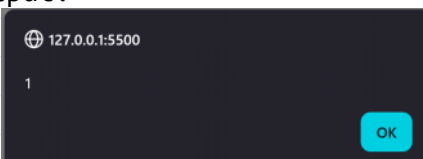
Are objects ordered? In other words, if we loop over an object, do we get all properties in the same order they were added? Can we rely on this?

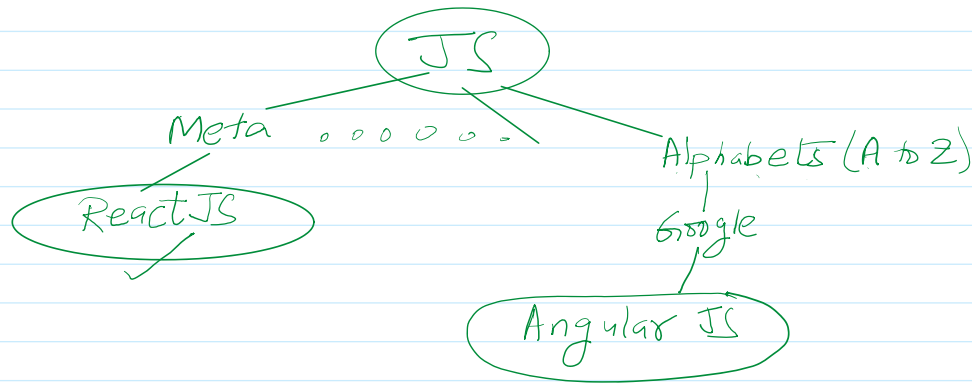
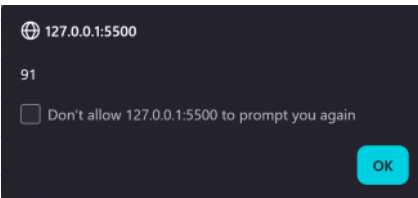
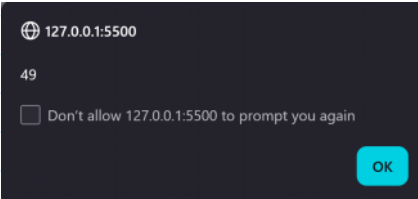
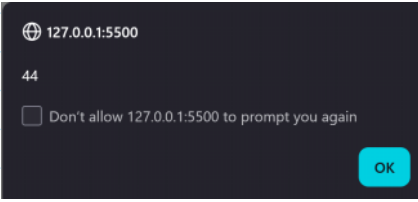
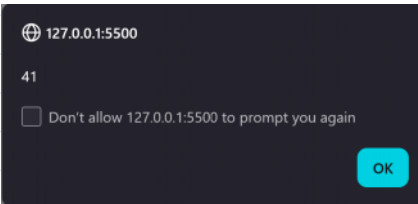
The short answer is: "ordered in a special fashion": integer properties are sorted. Other appears in creation order. The details follow.

As an example, let's consider an object with the phone codes.

```
let codes = {  
  "91": "India",  
  "1": "USA",  
  "49": "Germany",  
  "41": "Switzerland",  
  "44": "Great Britain"  
};  
for(let code in codes){  
  alert(code); //1, 41, 44, 49, 91  
}
```

Output:





Note: Integer properties? What's that?

The "integer property" term here means a string that can be converted to-and-from an integer without a change.

So "49" is an integer property name, because when it's transformed to an integer number and back, it's still the same. But "+49" and "1.2" are not.

Object references and copying

One of the fundamental differences of object versus primitive is that objects are stored and copied "by reference", whereas primitive values, strings, numbers, boolean, etc. are always copied as a whole value.

That's easy to understand if we look a bit under the hood of what happens when we copy a value.

Let's start with a primitive such as String.

```
let message = "Hello";  
let phrase = message;
```

As a result we have two independent variable, each one storing the string "Hello".

message "Hello"

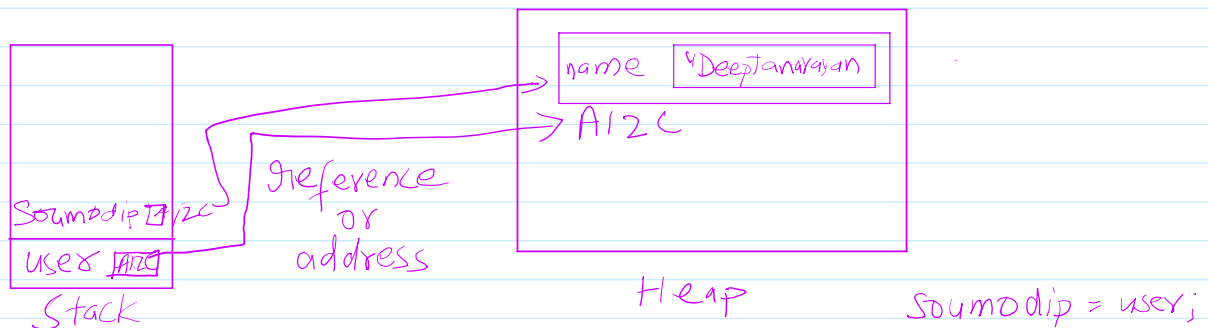
message "Hello"
phrase "Hello"

Quite an obvious result, right?

Objects are not like that.

A variable assigned to an object stores not the object itself, but its "address in memory"- In other words, "a reference" to it.

```
let user = {  
  name: `Deeptanarayan`,  
};
```



The object is stored somewhere in memory (at the right of the picture) while the user variable at the left has a "reference" to it.

We may think of an object such as user like a sheet of paper with the address of the object on it.

When we perform action with the object example take a property user.name, the Javascript engine looks at what's at that address and performs the operation on the actual object.

Now here's why it's important.

When an object is copied, the reference is copied, but the object itself is not duplicated.

For instance:

```
let user = {  
  name: `Deeptanarayan`,  
};  
let admin = user; //copy the reference
```

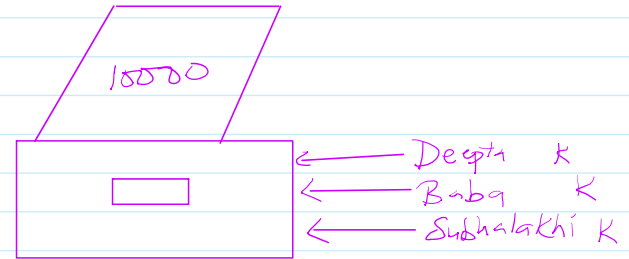
Now we have two variables, each storing the reference to the same object.

```
let user = {
```

```

    name: `Deeptanarayan`,
  };
  let admin = user; //copy the reference
  console.log(`User-name: ${user.name}`);
  console.log(`admin: ${admin.name}`)
  admin.name = `Pete`;
  console.log(`User-name: ${user.name}`)

```



Output:

```

User-name: Deeptanarayan
admin: Deeptanarayan
User-name: Pete

```

object variable/reference variable holds the address (key) to the memory, where actual data is stored.

It's as if we had a cabinet with two keys and used one of them (admin) to get into it and make changes. Then if we later use another key (user.) We are still opening the same cabinet and can access the changed contents.

Comparison by reference

Two objects are equal only if they are the same object.

For instance, here A and B reference the same object, thus they are equal.

```

→ let a = {};
→ let b = a; //copy the reference
console.log( a == b ); //true, Both variables reference the same object.
console.log(a === b); //true

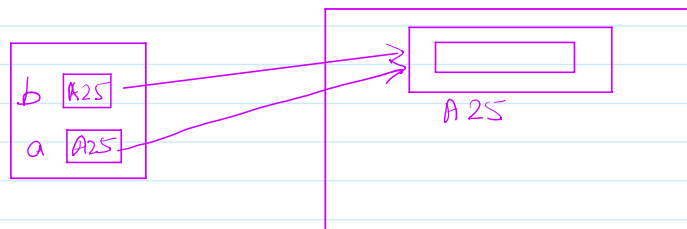
```

Output:

```

true
true

```



And here two independent objects are not equal even they do, they look alike (both are empty):

```

let a = {};
let b = {};
console.log( a == b ); //false, Both variables reference is not same.
console.log(a === b); //false

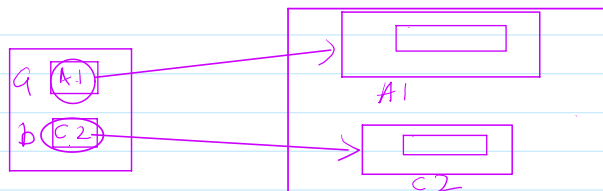
```

Output:

```

false
false

```



Const objects can be modified

An important side effect of storing object as reference is that an object declared as construction can be modified.

For instance:

```
const user = {  
  name: `Deeptanarayan`,  
};  
user.name = `Deeptanarayan Sarkar`; //(*)  
console.log(`${user.name}`)
```

Output:

Deeptanarayan Sarkar

It might seem that the line (*) would cause an error. But it does not. The value of user is constant. It must always reference the same object, but properties of that object are free to change.

In other words, the construction user gives an error only if we try to set user equal to... as a whole.

That said, if we really need to make constant object properties, it's also possible, but using totally different methods.